Using Physics-Informed Neural Networks for pricing derivative financial contracts under stochastic volatility

Stanislav Borisov

HSE University, Sber

October 14, 2026

Outline

1. Theory of the derivatives pricing with PDEs
2. Physics-Informed Neural Networks
3. Practical results
4. Conclusion and extensions

Using PDEs to price financial instruments

- As was established in **Fischer Black, Myron Scholes - The Pricing of Options and Corporate Liabilities (1973)**, the fair value of any financial contract can be represented by the following Partial Differential Equation:

$$\frac{\partial V}{\partial \tau} = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV$$

- Note, however, that this is a general expression, and to get a fair price of particular derivative, one must apply specific initial and boundary conditions, and sometimes modify the expression to also include Linear Complementarity Problem.

Solving Black-Scholes PDE

- For example, consider a European Call Option. It has terminal and boundary conditions:

$$V(0, S) = \max\{S - K, 0\}$$
$$V(\tau, S \to \infty) = S - Ke^{-r\tau}$$
$$V(\tau, S \to 0) = 0$$

- This problem can be solved analytically, and it yields famous Black-Scholes Formula:

$$C(S, \tau) = S\,N(d_1) - Ke^{-r\tau}\,N(d_2)$$
$$d_1 = \frac{\ln(S/K) + \left(r + \sigma^2/2\right)(\tau)}{\sigma\sqrt{\tau}}$$
$$d_2 = d_1 - \sigma\sqrt{\tau}$$

Stochastic volatility

- In reality, however, Black-Scholes model is too basic to reflect the market conditions accurately. Main problem - so called "volatility smile", that BS does not allow.
- One of the ways to account for that empirical effect - stochastic volatility (volatility of the price process is its own stochastic process). Most popular stochastic volatility model is Heston, which has the following PDE:

$$\frac{\partial u}{\partial \tau} = \frac{1}{2} S^2 v \frac{\partial^2 u}{\partial S^2} + \rho \sigma S v \frac{\partial^2 u}{\partial S \partial v} + \frac{1}{2} \sigma^2 v \frac{\partial^2 u}{\partial v^2} + rS \frac{\partial u}{\partial S} + \kappa(\theta - v) \frac{\partial u}{\partial v} - ru.$$

- As one can see, the problem's dimensionality increases compared to BS.

Numerical methods for Pricing PDE

- Call options under Heston have only quasi-analytical solution, and more complex instruments under this model cannot be solved analytically.
- Two standard approaches for solving such advanced problems are Monte-Carlo simulations and grid-based solvers for PDEs.
- Both have major downsides: slow convergence ($\mathcal{O}(1/\sqrt{n})$) for MC, curse of dimensionality for grid-based PDE solvers.
- My work is focused on developing and testing new approach for pricing derivatives with special kind of Neural Network.

Physics-Informed Neural Networks

- **M. Raissi, P. Perdikaris, G.E. Karniadakis - Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations (2019)** introduced Physics-Informed Neural Networks (PINNs).
- This architecture allows us to train neural networks on physical dependencies and (optionally) empirical data.

Problem setup

- Let V be a neural network which models the solution to a particular PDE.
- Given PDE of the general form $f\left(t, \mathbf{x}, u, \frac{\partial u}{\partial t}, \nabla u, \nabla^2 u \ldots, \lambda\right) = 0$, the residual of the network V is

$$r_V = f\left(t, \mathbf{x}, V, \frac{\partial V}{\partial t}, \nabla V, \nabla^2 V \ldots, \lambda\right)$$

- Partial derivatives with respect to model inputs can be obtained via autodiff.
- Then, the problem comes down to writing down a reasonable Loss function for the Network to learn.

Network learning problem

Setting loss function

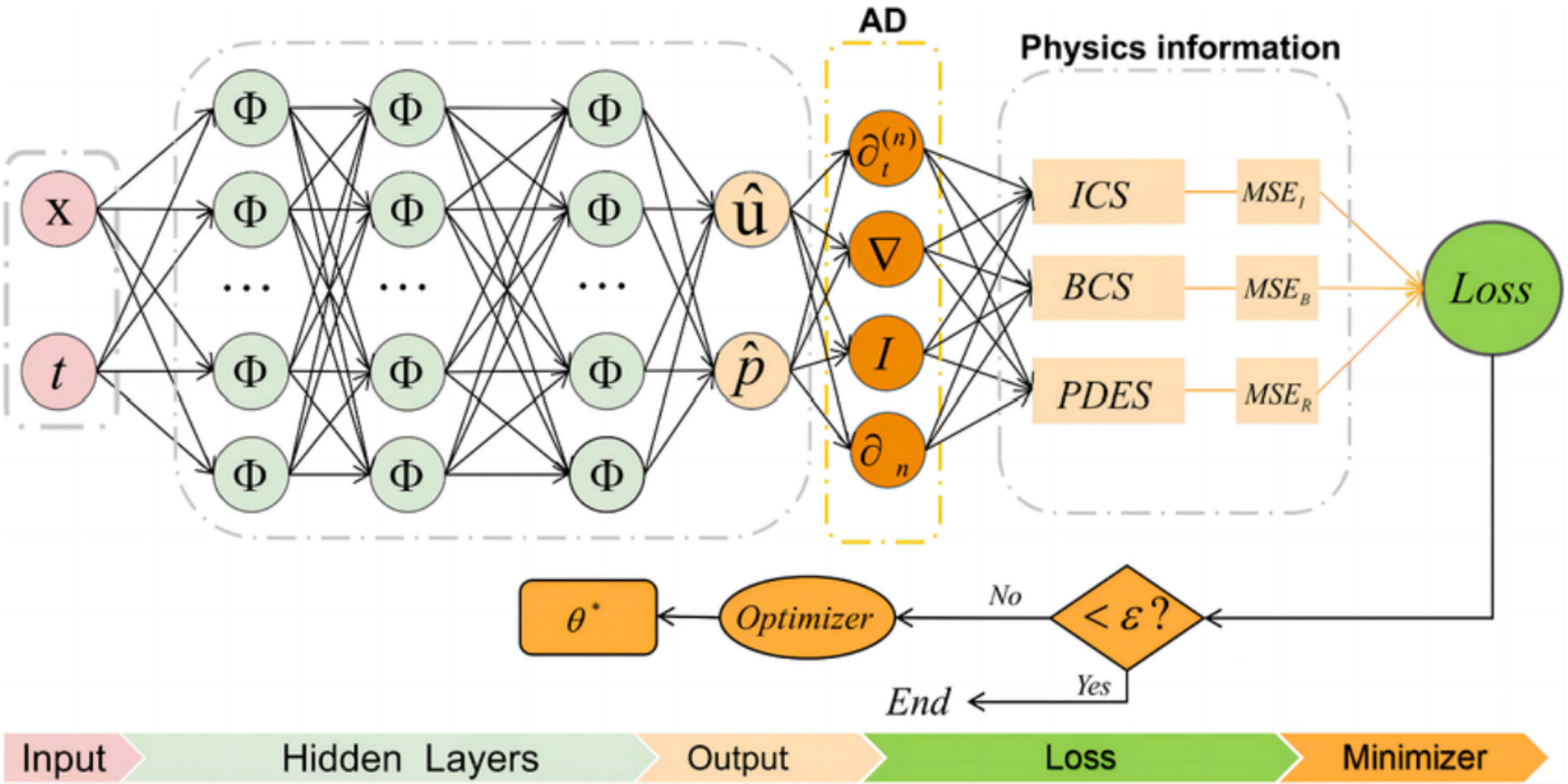
Define the loss function as:

$$\mathcal{L}_{total} = \mathcal{L}_{pde} + \mathcal{L}_{boundary} + \mathcal{L}_{initial} + \mathcal{L}^*_{data} + \mathcal{L}^*_{lcp}$$

Where:

- **PDE Loss:** The norm of the solution residual.
- **Boundary Conditions:** The norm of the difference between boundary conditions and predicted values on boundaries.
- **Initial:** The norm of the difference between initial conditions and predicted values on boundaries.
- **Data Loss:** Additionally, we can add the loss which punishes the divergence from the empirical data (if such data is available).
- **Linear Complementarity Problem Loss:** Optimal stopping loss.

The scheme of the learning



Variable transformation

- We can reduce the dimensionality of the problem by transitioning to log-moneyness defined as $x = \log(\frac{S}{K})$ and $u = \frac{V}{K}$. Then, the Heston PDE changes to:

$$u_\tau - \frac{1}{2} v\, u_{xx} - \left(r - \frac{1}{2} v\right) u_x - \rho\sigma v\, u_{xv} - \\ -\frac{1}{2}\sigma^2 v\, u_{vv} - \kappa(\theta - v)\, u_v + ru = 0$$

- This also transforms boundary and terminal conditions. For Calls, they become:

$$\varphi_{\text{call}}(x) = \max(e^x - 1, 0),$$
$$u_{\text{call}}(x_{\min}, \tau, r, \sigma) \approx 0,$$
$$u_{\text{call}}(x_{\max}, \tau, r, \sigma) \approx e^{x_{\max}} - e^{-r\tau}$$

BS baseline + neural correction

We can help the model learn even further: do not learn the full price, learn only a correction U on top of a closed-form Black-Scholes baseline.

$$u(x, v, \tau; \vartheta) \;=\; u_{\mathrm{BS}}\big(x, \tau;\, r, \sigma_{\mathrm{eff}}(v, \vartheta, \tau)\big) \;+\; U_\theta(x, v, \tau; \vartheta)$$

- u_{BS}: analytical BS price
- U_θ: PINN correction for stochastic-vol effects
- PDE residual enforced on the *total* price $u_{\mathrm{BS}} + U_\theta$

Why it helps

Because we "de-scale" the function the Network learns, adjustment $\to 0$ OTM and ITM.

Comparison with current pricing methods

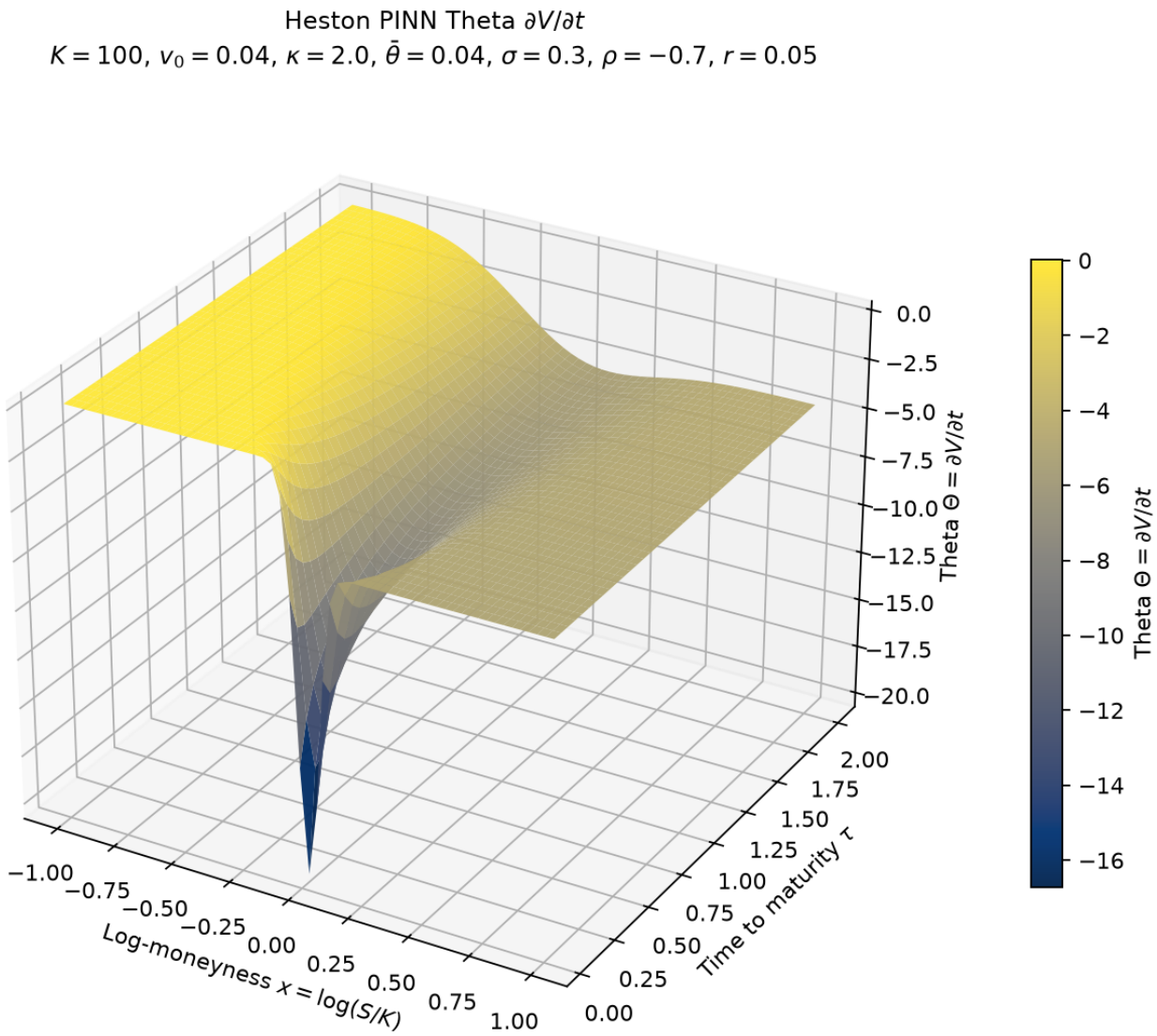
- Such alterations to the original equation lead to significant performance boost (Heston, errors in normalized price $u = V/K$, vs. COS):

Table: Accuracy and runtime for parametric Heston pricing.

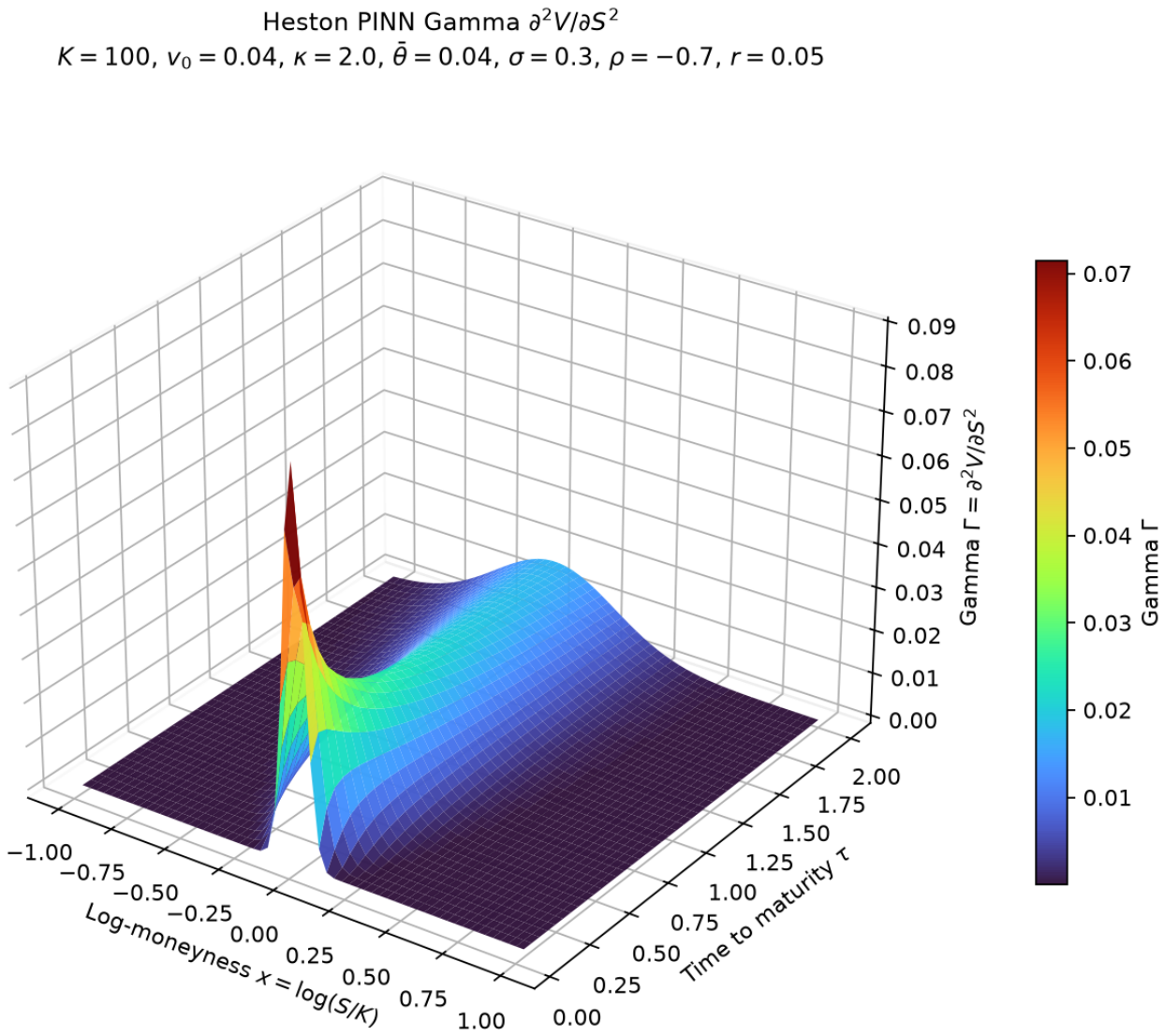
Method	RMSE	Max err.	Train	Inf. time
PINN	0.002	0.003	9494 s	151 μs
FD	0.002	0.005	[n/a]	1.5×10^7 μs
MC	0.006	0.011	[n/a]	21000 μs

- PINNs have high up-front costs, but really fast evaluation speeds once trained. Hence, PINNs are preferred when frequent evaluations of large portfolios of deals are required.

Qualitative results - Theta ($\partial V / \partial \tau$)



Qualitative results - Gamma ($\partial^2 V / \partial S^2$)



Advantages of PINNs

This approach unlocks multiple possibilities for making financial markets more efficient.

- With quick pricing, real-time re-evaluation of even large portfolios of derivatives under advanced models is possible. This opens new opportunities for their risk management.
- PINNs make the trading of even most complex products accessible to smaller financial institutions (as trained models do not consume lots of compute).

Further work

There is a number of ways I can extend my current research:

- Using PINNs to solve inverse problem: given observable market prices, calibrate the parameters of the model.
- Applying PINNs to more complex products, which have no analytical pricing.

Conclusion

Thank you!
I am ready to answer any of your questions.